



Ministerio de Educación
Colegio Ingeniero Tomás Guardia
WRO - Futuros Ingenieros
Diario de Ingeniería
Sentinels

Integrantes : Makenzie Agrazal Velásquez 12•

David Garibaldy 11•

Cesar Yau Feng 10•

Tutora: Carolina Gamarra



Introducción

El presente Diario de Ingeniería documenta el proceso de diseño, construcción, programación, prueba, diagnóstico y mejora del robot del nuestro equipo Sentinels para la categoría Futuros Ingenieros de la WRO 2026. El objetivo de esta documentación es reconstruir el razonamiento que llevó al equipo desde un prototipo inicial hasta una plataforma más estable y preparada para afrontar las condiciones de competencia.

Durante el proyecto entendimos que un robot autónomo no puede evaluarse como una suma de piezas independientes. El chasis condiciona la estabilidad; la estabilidad afecta las lecturas de los sensores; las lecturas alimentan la lógica de software; y la lógica determina la actuación de los motores y del servomotor. Por ello, cada modificación se trató como una intervención sobre un sistema completo.

El desarrollo tuvo varias etapas: construcción y ajuste del chasis, calibración de tracción y dirección, integración de sensores ultrasónicos, depuración del cableado y del programa, filtrado de mediciones, control de giros y vueltas, incorporación experimental de visión mediante HuskyLens, diseño de una base impresa en 3D para ese dispositivo, los sensores ultrasónicos y, finalmente, corrección específica del comportamiento del robot en sentido horario.

Una decisión importante surgió durante la competencia regional: el equipo no pudo disponer de la cámara prevista. En lugar de detener el desarrollo, se decidió mantener una estrategia funcional sin cámara y continuar utilizando los sensores disponibles. Esta experiencia reforzó uno de los principios centrales del proyecto: diseñar con tolerancia a cambios de hardware y mantener una solución capaz de operar cuando un componente previsto no está disponible.

Objetivos

1. Construir un carrito autónomo capaz de desplazarse de manera estable dentro de la pista.
2. Desarrollar una arquitectura mecánica suficientemente rígida para soportar las pruebas repetidas.
3. Integrar 2 ultrasonicos y procesar sus lecturas para tomar decisiones de movimiento.
4. Calibrar dirección, velocidad y giros para conseguir un balance entre rapidez y precisión.
5. Implementar una lógica de finalización autónoma mediante el conteo de giros/vueltas.
6. Explorar la incorporación de HuskyLens como alternativa o complemento de percepción.
7. Documentar la construcción y ajustes de ingeniería de forma que otro equipo pueda comprender y reproducir el sistema.

Índice

1. Introducción	2
2. Objetivos	3
3. Metodología	5
4. Movilidad y diseño	6
4.1. Características del primer diseño	6
4.2. Sistema de dirección	7
4.3. Sistema de tracción	7
4.4. Problemas encontrados en el primer diseño	8
4.5. Evolución del diseño mecánico	9
4.6. Distribución de componentes	10
4.7. Pruebas de movilidad	11
4.8. Mejoras para la etapa nacional	13
5. Materiales utilizados	14
5.1. Estructura y movilidad	14
5.2. Electrónica y control	14
5.3. Sistema de visión	14
5.4. Piezas fabricadas en 3D	14
6. Evidencia documental y fotográfica	16
7. Registro de resultados	27
8. Desarrollo del Código	29
8.1. Pseudocódigo inicial (30 de mayo)	30
8.2. Primera estrategia de navegación	31
8.3. Refinamiento de la lógica (4 de junio)	36
8.4. Implementación en Arduino (14 de junio)	39
8.5. Calibración de velocidad	45
8.6. Integración de HuskyLens (7 - 16 de agosto)	49
8.7. Código final integrado	55
9. Impresión 3D.....	58
10.Huskylends.....	61
11. Organización del equipo.....	71
12. Conclusión.....	75

Metodología

La metodología de trabajo de nuestro equipo se basó en un proceso iterativo de diseño, construcción, programación, prueba, diagnóstico y mejora continua. El robot fue considerado como un sistema completo, en el que los componentes mecánicos, electrónicos y de software están relacionados entre sí.

El proceso comenzó con el análisis y planificación de los objetivos, seguido de la construcción del prototipo inicial y la integración de sus principales componentes, como Arduino, controlador de motores, servomotor y sensores ultrasónicos. Posteriormente, se desarrolló el programa de control y se realizaron pruebas en la pista para calibrar la dirección, velocidad y lecturas de los sensores.

Durante las pruebas se identificaron diferentes problemas, como desviaciones en la trayectoria y lecturas inestables. Para solucionarlos se realizaron procesos de diagnóstico, pruebas de diferentes soluciones y ajustes tanto mecánicos como electrónicos y de programación. También se diseñaron e imprimieron en 3D soportes para mejorar la estabilidad de los sensores y otros componentes.

En una etapa posterior se experimentó con la integración del sistema de visión HuskyLens, diseñando una base 3D, programando la detección de colores y combinando esta función con el sistema de navegación mediante sensores ultrasónicos.

Finalmente, se realizaron pruebas de validación para comprobar la navegación autónoma, los giros, el conteo de vueltas y la detención automática. La experiencia durante la competencia regional, en la que no se pudo disponer de la cámara prevista, llevó al equipo a mantener una estrategia alternativa utilizando los sensores disponibles. De esta manera, se buscó desarrollar un robot estable, funcional y capaz de adaptarse a cambios en sus componentes.

Movilidad y diseño

Durante las primeras etapas del proyecto, el equipo decidió utilizar una estructura de cuatro ruedas montada sobre un chasis rígido. Esta configuración fue seleccionada buscando conseguir estabilidad durante el desplazamiento y suficiente superficie de contacto con la pista.

El primer modelo utilizado durante las dos competencias regionales fue construido alrededor de un chasis de cuatro ruedas. Sobre esta estructura se instalaron los componentes electrónicos necesarios para controlar el movimiento y realizar la navegación autónoma.

En la parte central del chasis se ubicó el Arduino junto con los elementos de control, mientras que los sensores ultrasónicos fueron colocados en la parte frontal para obtener información sobre el entorno.

La distribución de los componentes fue realizada procurando mantener una estructura suficientemente estable para que el robot no cambiara excesivamente su trayectoria durante los recorridos.

4.1. Características principales del primer diseño

El primer prototipo contaba con:

- Chasis de cuatro ruedas.
- Cuatro ruedas de tracción/desplazamiento.
- Arduino Uno.
- Controlador de motores.
- Servomotor para el sistema de dirección.
- Dos sensores ultrasónicos utilizados para determinar el comportamiento del robot durante el recorrido.
- Soportes impresos en 3D para los sensores ultrasónicos.
- Protoboard para realizar las conexiones.
- Sistema de alimentación mediante 2 baterías de 3.7 V.
- Cableado para interconectar los diferentes componentes.

La configuración permitió realizar las primeras pruebas de movimiento y desarrollar la estrategia de navegación basada en los sensores ultrasónicos.

4.2. Sistema de dirección

El robot utiliza un servomotor como mecanismo de dirección. En lugar de controlar directamente la dirección mediante dos motores independientes, el servomotor modifica el ángulo de las ruedas de dirección.

Durante las primeras pruebas se estableció una posición central de:

90° → movimiento recto

Mientras que se utilizaron posiciones diferentes para realizar las correcciones:

50° → giro hacia la izquierda

130° → giro hacia la derecha

Estos valores no fueron seleccionados únicamente de manera teórica. Fueron probados físicamente durante las prácticas de pista y posteriormente ajustados según la respuesta real del robot.

La utilización de ángulos definidos permitió que el código pudiera asociar una acción específica con cada posición del servomotor.

Por ejemplo:

90° → avanzar recto

50° → girar izquierda

130° → girar derecha

Esta configuración permitió mantener una lógica sencilla y reproducible entre las diferentes pruebas.

4.3. Sistema de tracción

Para el desplazamiento se utilizó un sistema de motor controlado mediante un driver de motores, permitiendo regular la velocidad desde el programa.

La velocidad se controló mediante PWM, utilizando valores definidos en el código:

```
int velRecta = 220;
```

```
int velCurva = 220;
```

Inicialmente se probaron diferentes valores de velocidad para determinar cuál permitía obtener el mejor equilibrio entre rapidez y estabilidad.

Durante las pruebas se utilizaron valores de:

255 → 180 → 170 → 220

El valor 255 permitía una velocidad mayor, pero durante los giros se observaron pequeñas desviaciones y una menor capacidad de corrección.

Posteriormente se redujo la velocidad para observar si el comportamiento mejoraba. Después de comparar los resultados de las pruebas, determinamos que 220 era el valor más funcional para nuestro robot.

Por esta razón, la velocidad de 220 se mantuvo como referencia para las

Uno de los aspectos que tuvimos que analizar durante las pruebas fue que la velocidad no podía considerarse de manera independiente del sistema de dirección.

El diseño de los soportes también permitió mantener los sensores en una posición relativamente fija durante el movimiento.

Durante las pruebas observamos que pequeños cambios en la posición de un sensor podían afectar las mediciones. Por esta razón, se consideró importante que los sensores no quedaran sueltos o cambiaran de orientación después de varios recorridos.

Esto llevó al equipo a utilizar soportes impresos en 3D diseñados específicamente para el robot.

4.4. Problemas encontrados en el primer diseño

El primer diseño permitió desarrollar la estrategia inicial, pero las pruebas también permitieron identificar diferentes aspectos que podían mejorarse.

Entre los principales problemas observados estuvieron:

- Pequeñas desviaciones durante el desplazamiento recto.
- Diferencias en el comportamiento de los giros.
- Variaciones en las lecturas de los sensores.
- Movimiento de algunos elementos después de varios recorridos.
- Necesidad de mejorar la ubicación y estabilidad de los sensores.
- Necesidad de encontrar una velocidad adecuada.
- Necesidad de mantener los componentes correctamente sujetos al chasis.

Estos problemas no fueron considerados únicamente como fallas, sino como información para realizar modificaciones al diseño.

4.5. Evolución del diseño mecánico

Después de las primeras competencias regionales, el equipo continuó trabajando en el robot con el objetivo de prepararlo para la etapa nacional.

La principal diferencia entre el modelo utilizado anteriormente y el modelo destinado a la final es la incorporación de nuevos elementos y una reorganización de la estructura para integrar el sistema de visión.

La incorporación de HuskyLens generó una nueva necesidad mecánica: no era suficiente colocar la cámara sobre el robot de manera provisional. Era necesario mantenerla en una posición estable para que pudiera realizar las pruebas de reconocimiento de manera consistente.

Por esta razón, se diseñó una base específica en 3D para HuskyLens.

Base 3D para HuskyLens

La base de HuskyLens fue diseñada para solucionar una necesidad que surgió al incorporar el sistema de visión.

La cámara debía mantenerse:

- estable;
- correctamente posicionada;
- protegida durante el movimiento;
- integrada con la estructura del robot.

El diseño 3D permitió crear una pieza adaptada específicamente a la configuración del robot.

Esto representa una modificación mecánica relacionada directamente con una necesidad del sistema electrónico y de software: la posición de la cámara influye en la información que recibe el sistema de visión.

Por esta razón, el diseño de la base no se realizó únicamente por estética, sino como una solución funcional para mejorar la integración de HuskyLens.

Modelo utilizado para la etapa final

Para la etapa nacional se mantuvo la filosofía de cuatro ruedas del modelo anterior, pero se realizaron modificaciones para adaptar el robot a las nuevas necesidades del proyecto.

La estructura final incorpora:

- Chasis de cuatro ruedas.
- Arduino Uno.
- Controlador de motores.
- Dos sensores ultrasónicos.
- Servomotor.
- Protoboard.
- Sistema de alimentación.
- Soportes 3D para los sensores.
- HuskyLens.
- Soporte 3D para HuskyLens.
- Base 3D para la protoboard.
- Cableado reorganizado para integrar los nuevos componentes.

La incorporación de HuskyLens aumentó la cantidad de elementos que debían ubicarse sobre el chasis. Por esto, la distribución física de los componentes se convirtió en una parte importante del proceso de diseño.

4.6. Distribución de componentes

La distribución de los componentes se realizó procurando evitar que los elementos electrónicos interfirieran con las ruedas, la dirección o los sensores.

La estructura puede dividirse conceptualmente en tres zonas:

Zona frontal

En esta zona se encuentran principalmente los elementos relacionados con la percepción:

- Sensores ultrasónicos.
- HuskyLens.
- Soportes 3D.

Esta ubicación permite que los dispositivos tengan una visión hacia la parte delantera del robot.

Zona central

En la zona central se encuentran los componentes principales de control:

- Arduino Uno.
- Protoboard.
- Conexiones.
- Parte del cableado.

La ubicación central ayuda a mantener los componentes protegidos dentro de la estructura y facilita el acceso durante las modificaciones.

Zona de alimentación y control

En esta zona se encuentran:

- Porta baterías.
- Baterías.
- Controlador de motores.
- Conexiones de alimentación.

La distribución se realizó buscando evitar que el peso quedara concentrado completamente en un solo extremo.

4.7.Pruebas de movilidad

La movilidad del robot fue validada mediante pruebas repetidas en pista.

Durante las pruebas se observaron principalmente:

1. Capacidad de avanzar recto.
2. Capacidad de realizar giros.
3. Tiempo necesario para completar una curva.
4. Estabilidad después de un giro.
5. Respuesta ante las paredes.
6. Comportamiento de los sensores durante el desplazamiento.
7. Efecto de la velocidad sobre la trayectoria.
8. Capacidad de completar varias vueltas.
9. Capacidad de detenerse después de alcanzar el número de giros establecido.

Las pruebas permitieron comprobar que el diseño mecánico y la programación estaban directamente relacionados.

Por ejemplo, un cambio en el ángulo del servomotor podía modificar la trayectoria, mientras que un cambio en la velocidad podía alterar la precisión del giro.

Por esta razón, las modificaciones mecánicas no se evaluaron únicamente con el robot detenido, sino mediante pruebas reales de pista.

Pruebas de giro

Uno de los aspectos que más atención requirió fue el comportamiento durante los giros.

Inicialmente se probaron diferentes configuraciones para determinar qué ángulo permitía realizar una curva sin que el robot se desviara demasiado.

Finalmente se establecieron como valores de referencia:

Movimiento Ángulo

Recto 90°

Izquierda 50°

Derecha 130°

Estos valores fueron utilizados posteriormente en el código.

La configuración permitió mantener una relación directa entre la programación y el movimiento físico:

Decisión del programa → posición del servo → dirección de las ruedas → trayectoria del robot.

Estabilidad mecánica

La estabilidad fue otro factor considerado durante el desarrollo.

El robot debía transportar simultáneamente:

- electrónica;
- sensores;
- cableado;
- baterías;

- controlador;
- protoboard;
- servomotor;
- HuskyLens.

Por esta razón, fue necesario considerar la distribución de los componentes y evitar que elementos importantes quedaran sueltos.

Las piezas impresas en 3D permitieron solucionar algunas de estas necesidades al crear soportes específicos para componentes que no podían fijarse adecuadamente utilizando únicamente las piezas originales del chasis.

4.8. Mejoras realizadas para la etapa nacional

Después de utilizar el modelo anterior en las dos competencias regionales, el equipo aprovechó la experiencia obtenida para continuar desarrollando el robot.

Las principales mejoras fueron:

- Incorporación de HuskyLens.
- Diseño e impresión de una base 3D para HuskyLens.
- Incorporación de un soporte para la cámara.
- Diseño de una base 3D para la protoboard.
- Reorganización de componentes.
- Revisión del cableado.
- Ajuste de la estrategia de navegación.
- Corrección del funcionamiento del recorrido en sentido horario.
- Nuevas pruebas de pista.
- Integración de la cámara con la estrategia de navegación.

Estas modificaciones surgieron de las necesidades identificadas durante las etapas anteriores y no de cambios únicamente estéticos.

Materiales utilizados

5.1.Estructura y movilidad

Cantidad	Componente	Función
1	Chasis	Estructura principal del robot
4	Ruedas	Desplazamiento del robot
1	Servomotor	Control de dirección
1	Motor/mecanismo de tracción	Movimiento del robot
1	Driver de motores L298N	Control del motor
—	Tornillos, separadores y elementos de fijación	Ensamblaje

5.2.Electrónica y control

Cantidad	Componente	Función
1	Arduino Uno	Control principal
2	Sensores ultrasónicos	Medición de distancia
1	Protoboard	Distribución de conexiones
1	Base 3D para protoboard	Fijación de la protoboard
1 paquete	Cables macho-macho	Conexiones eléctricas
2	Porta baterías	Alimentación
4	Baterías de 3.7 V	Fuente de alimentación

5.3.Sistema de visión

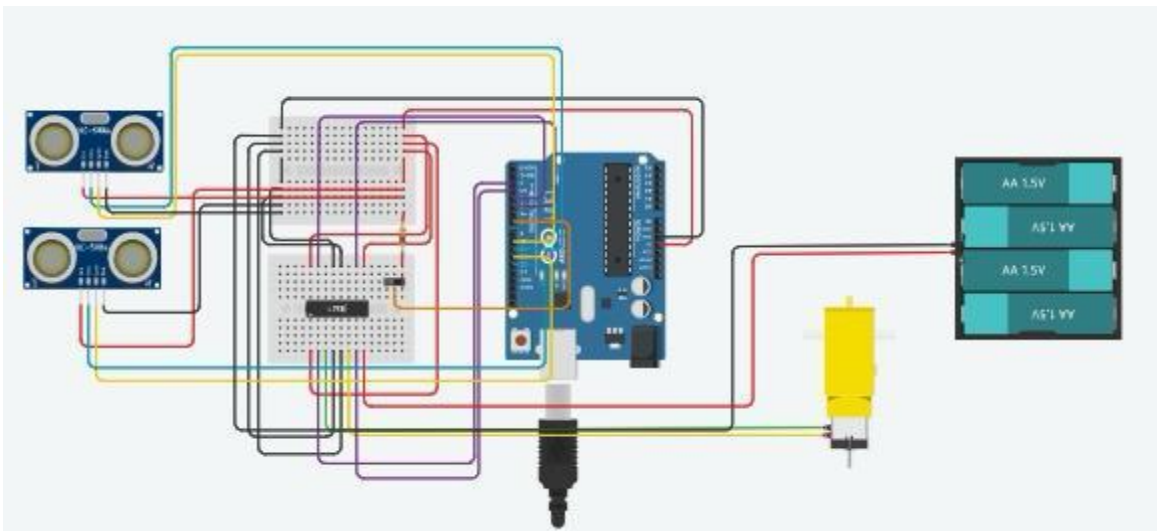
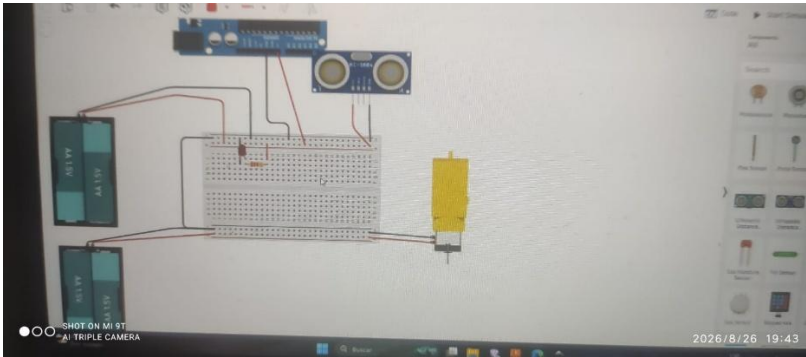
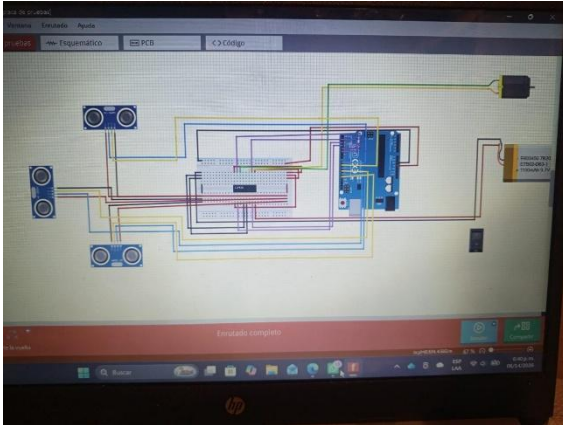
Cantidad	Componente	Función
1	HuskyLens	Detección mediante visión
1	Soporte para HuskyLens	Fijación de la cámara

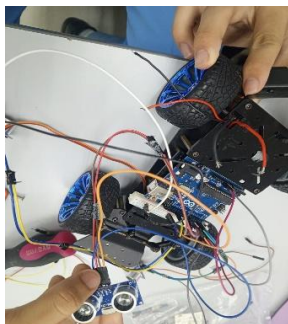
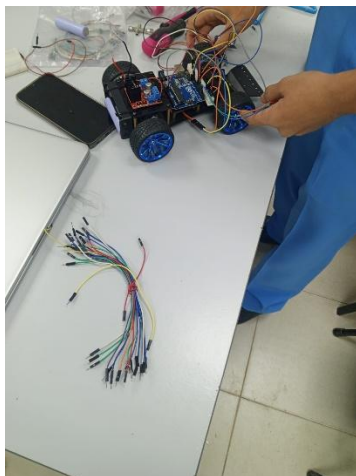
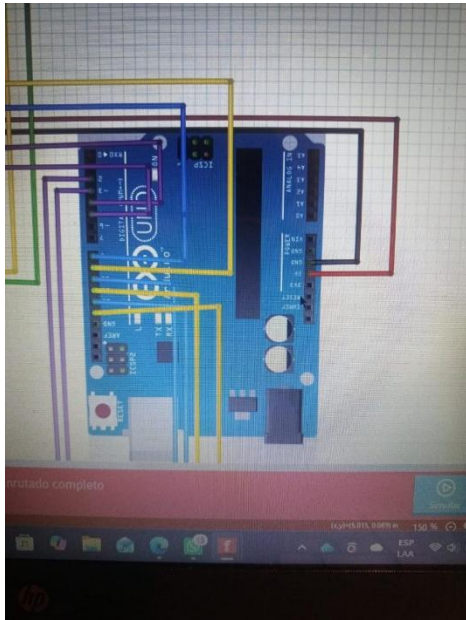
5.4.Piezas fabricadas mediante impresión 3D

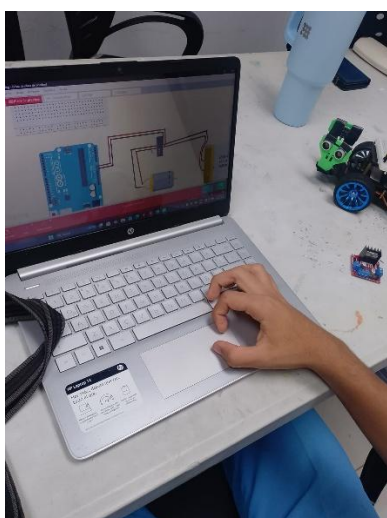
- 2 bases/soportes para los sensores ultrasónicos.
- 1 base para HuskyLens.

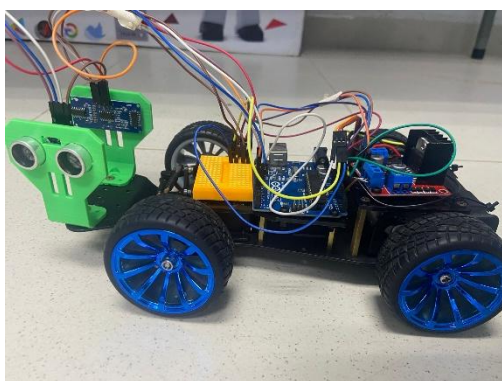
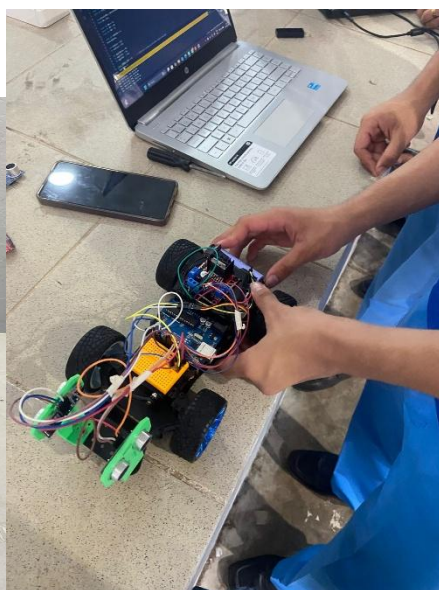
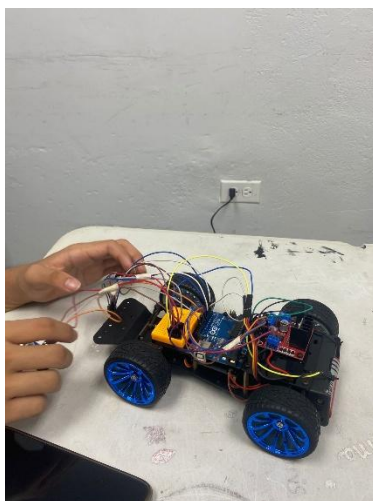
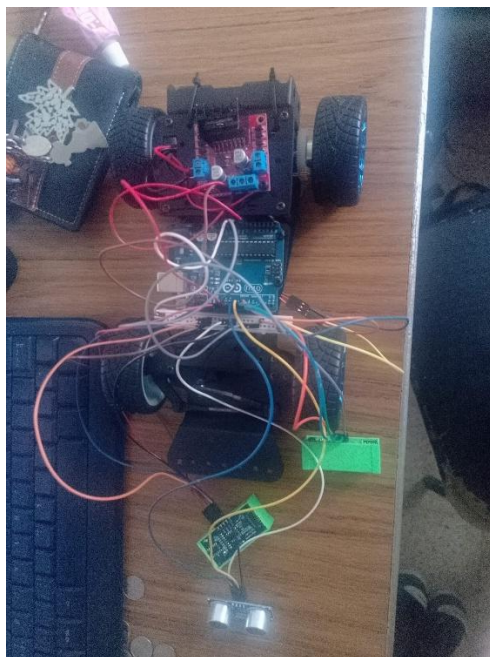
Evidencia documental y fotográfica

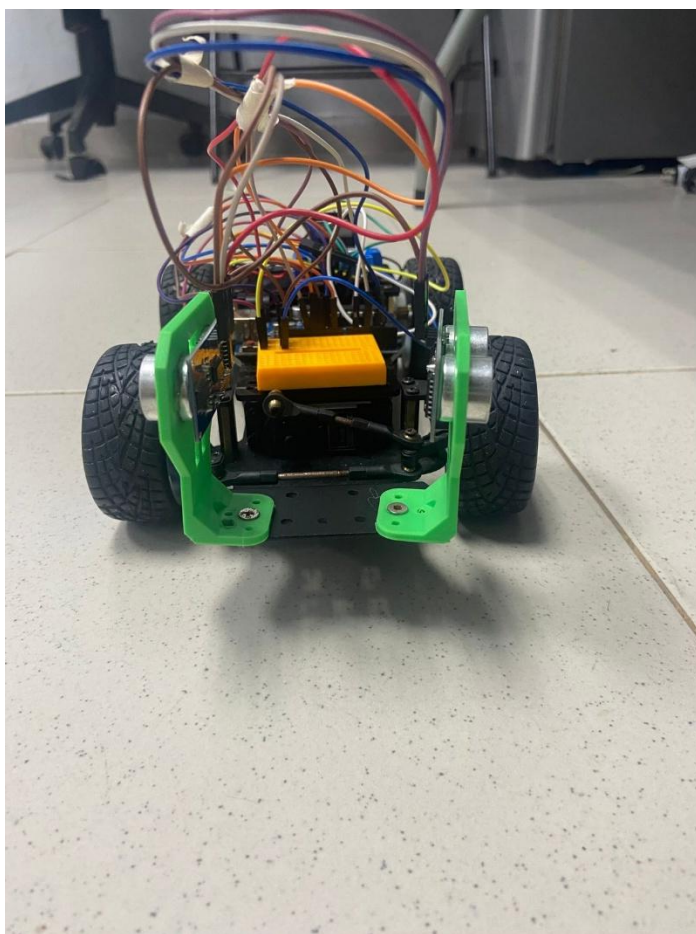
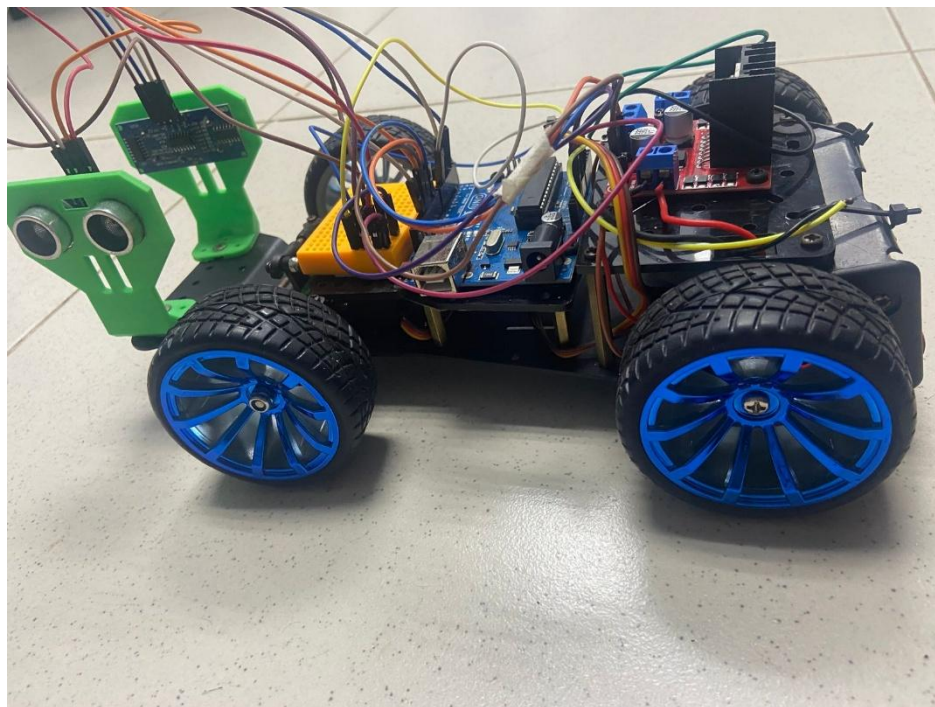
Esquema de Circuitos

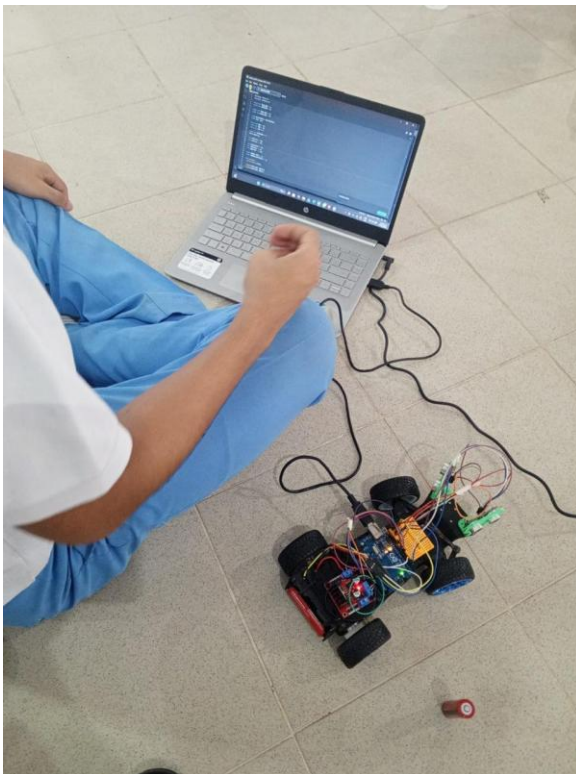
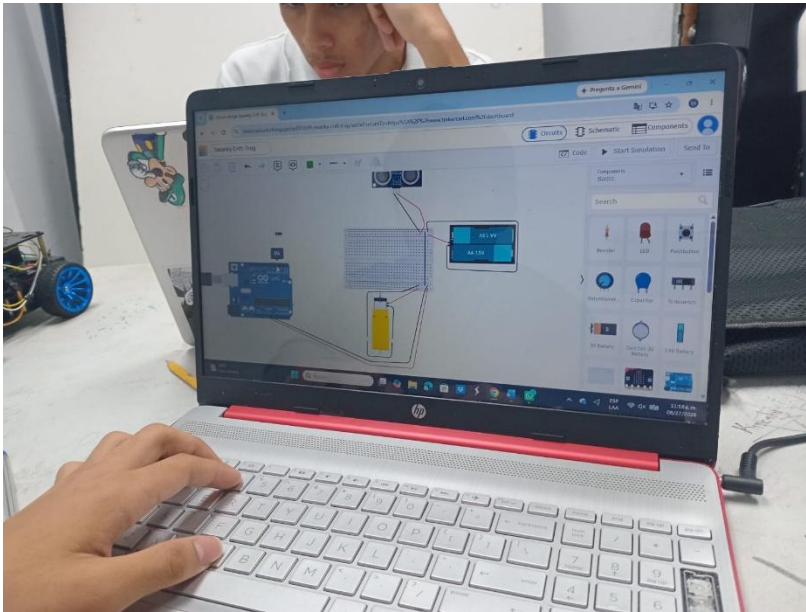


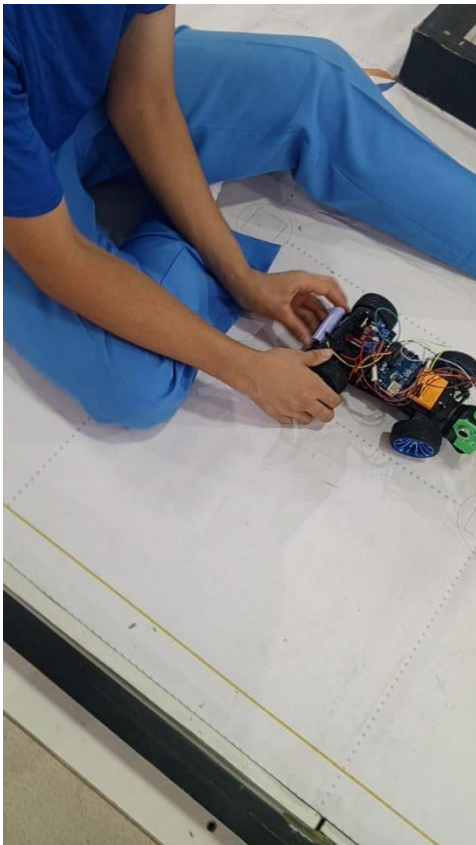


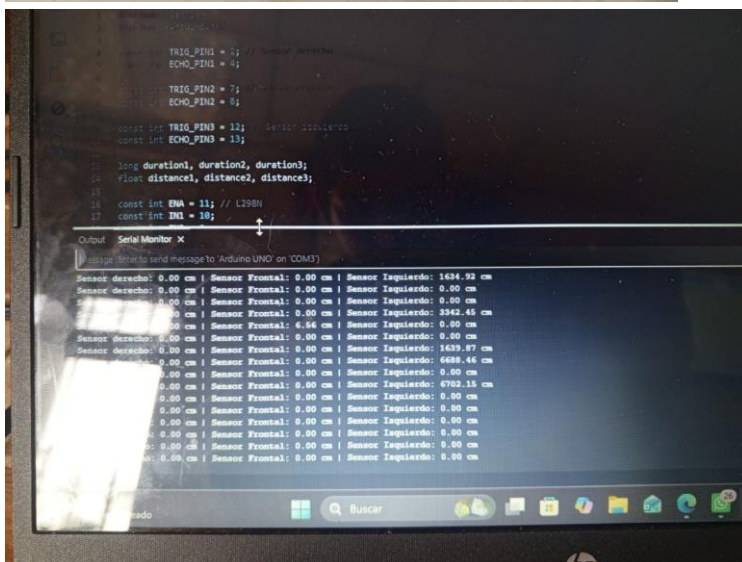


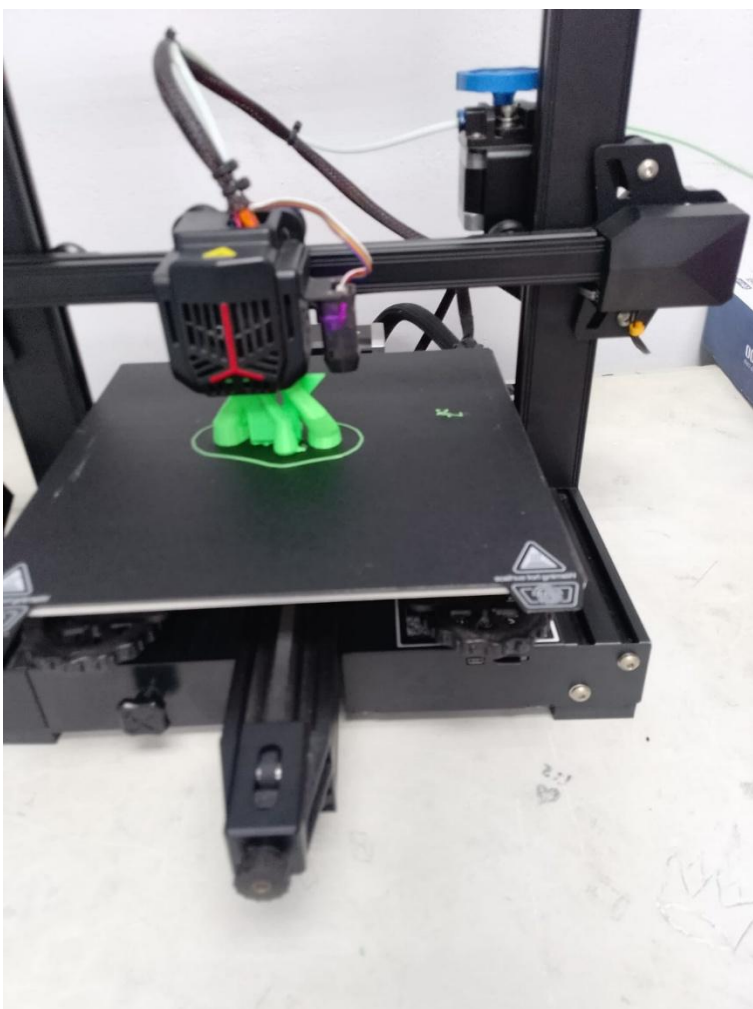
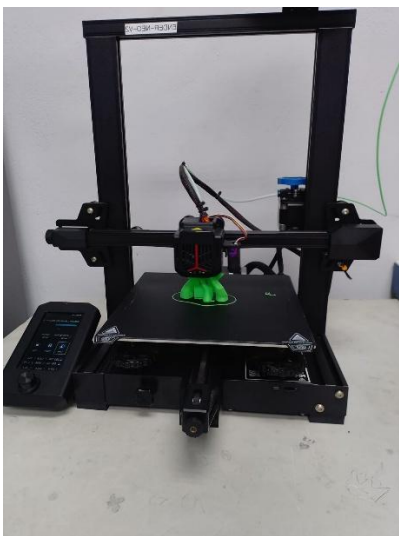


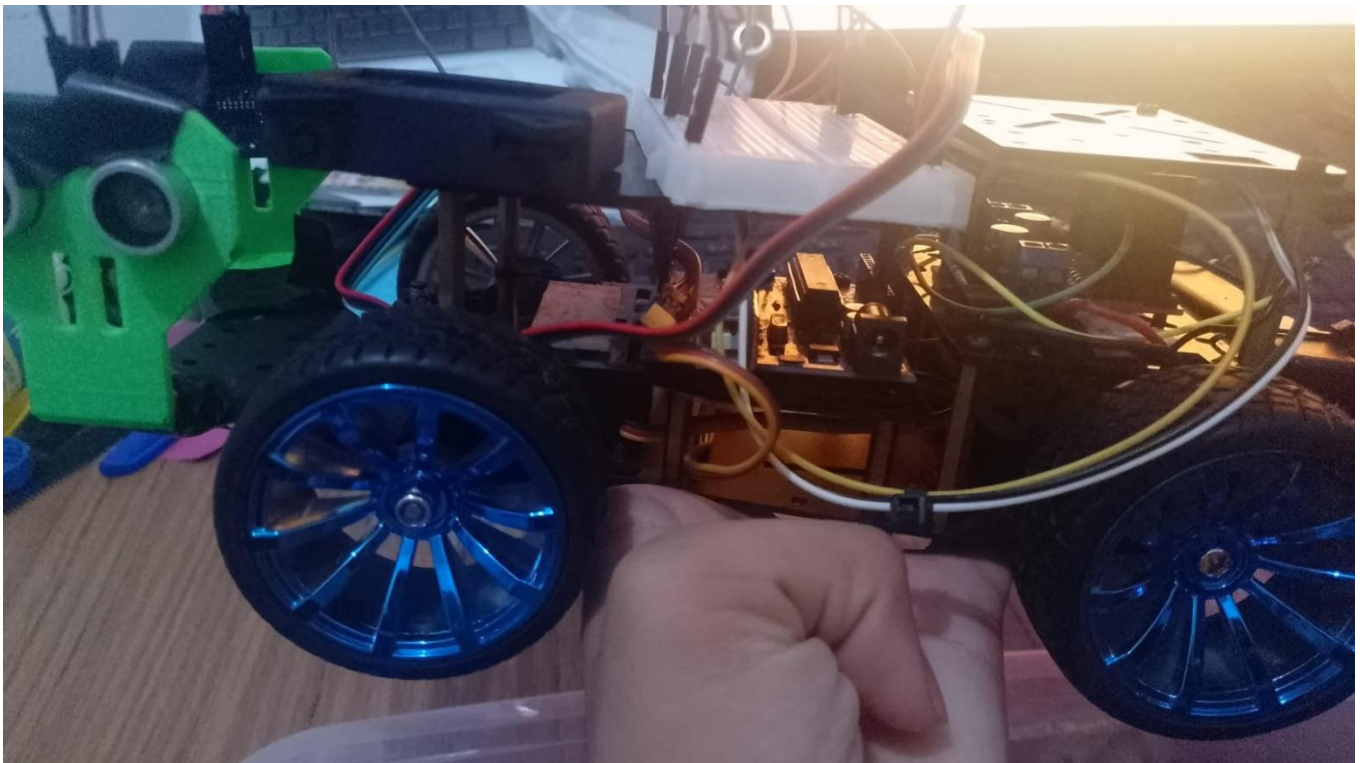


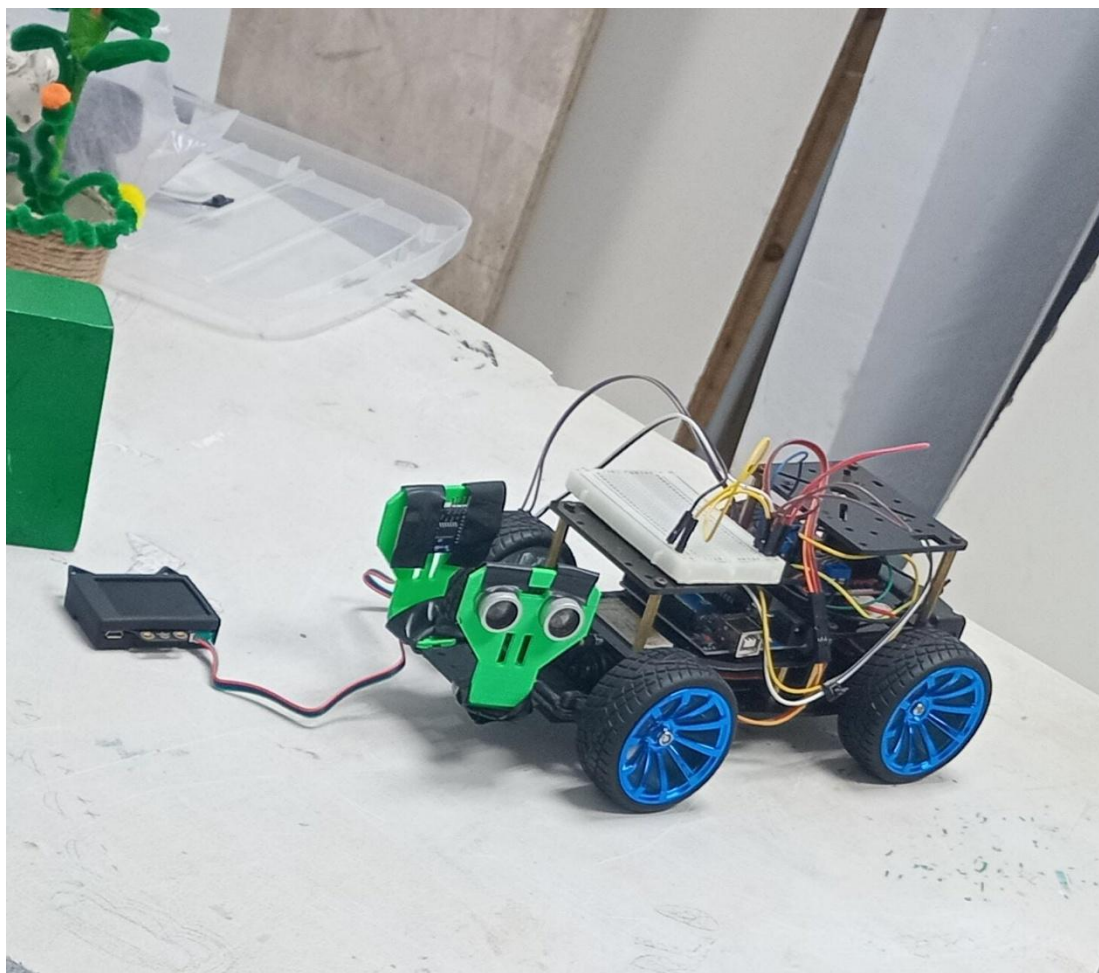
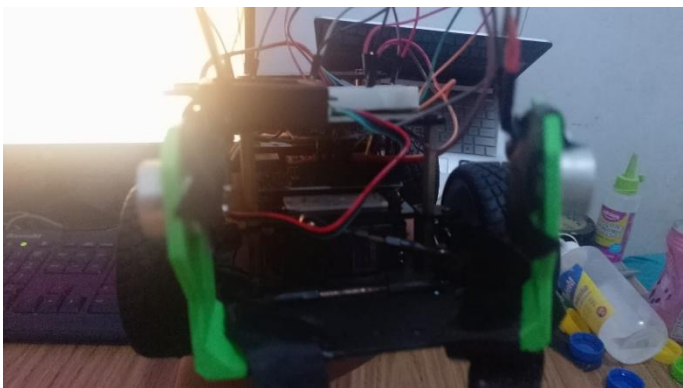












Registro de resultados

Área evaluada	Prueba realizada	Resultado	Estado
Movilidad	Avance recto	Se mejoró la estabilidad de la trayectoria	Validado
Dirección	Pruebas de ángulos del servo	50°, 90° y 130° establecidos como referencias	Validado
Velocidad	255, 180, 170 y 220	220 seleccionado como configuración funcional	Validado
Sensores ultrasónicos	Pruebas de lectura	Se identificaron variaciones y se realizaron ajustes	En desarrollo/validado parcialmente
Conteo de giros	Pruebas de recorrido	Se implementó contador de 12 giros	Validado
Soportes 3D	Pruebas de montaje	Se desarrollaron soportes para sensores	Validado
Base protoboard	Montaje	Protoboard fijada al chasis	Validado
HuskyLens	Integración física	Cámara incorporada mediante soporte/base 3D	Validado
HuskyLens	Prueba en pista	Todavía no realizada	Pendiente
Sentido horario	Pruebas de código	Se modificó la lógica para mejorar el recorrido horario	En validación

Sentido antihorario	Pruebas de recorrido	Se realizaron pruebas durante el desarrollo	Validado parcialmente
--------------------------------	-------------------------	---	-----------------------

Código

8.1. Primera etapa: desarrollo del pseudocódigo 30 de mayo

El 30 de mayo comenzamos a transformar la idea de funcionamiento del robot en un pseudocódigo. El objetivo de esta primera etapa fue establecer la lógica que posteriormente sería implementada en Arduino.

En esta primera versión se contemplaron tres sensores ultrasónicos: uno derecho, uno frontal y uno izquierdo. La función de los sensores era proporcionar información sobre las distancias alrededor del robot para permitirle tomar decisiones de movimiento.

También se definieron los principales componentes que debía controlar el programa:

- Arduino UNO.
- Controlador de motores.
- Motor de tracción.
- Servomotor de dirección.
- Tres sensores ultrasónicos.

Se establecieron además los pines correspondientes a cada componente y diferentes valores iniciales de velocidad.

Lógica de movimiento

La primera versión utilizaba varias funciones independientes:

- `moveForward()`
- `moveBack()`
- `stop()`
- `turnRight()`
- `turnLeft()`
- `leerSensores()`

La función `moveForward()` colocaba el servomotor en 90°, considerado como la posición recta, y activaba el motor para avanzar.

La función `moveBack()` hacía lo contrario en el controlador de motores para permitir que el robot retrocediera.

La función `stop()` detenía el motor colocando las salidas del controlador en estado bajo y estableciendo la velocidad en cero.

Para los giros, el servomotor se movía hacia posiciones extremas:

- $0^\circ \rightarrow$ izquierda
- $180^\circ \rightarrow$ derecha

En esta etapa todavía no buscábamos obtener el código definitivo. El objetivo era comprobar que podíamos expresar de manera ordenada las decisiones que tendría que tomar el robot.

8.2. Primera estrategia de navegación

La estrategia inicial estaba basada principalmente en el sensor frontal.

Cuando:

$cen < 15\text{ cm}$

el robot interpretaba que tenía un obstáculo demasiado cerca.

La secuencia programada era:

detenerse $\rightarrow$ esperar $\rightarrow$ retroceder $\rightarrow$ detenerse $\rightarrow$ analizar izquierda/derecha $\rightarrow$ girar.

La decisión del giro dependía de la comparación:

$izq > der$

Si había más espacio a la izquierda, el robot giraba hacia ese lado.

Si:

$der > izq$

el robot giraba hacia la derecha.

INICIO

DEFINIR TRIG_PIN1 COMO 2

DEFINIR ECHO_PIN1 COMO 4

DEFINIR TRIG_PIN2 COMO 7

DEFINIR ECHO_PIN2 COMO 8

DEFINIR TRIG_PIN3 COMO 12

DEFINIR ECHO_PIN3 COMO 13

DEFINIR ENA COMO 11

DEFINIR IN3 COMO 10

DEFINIR IN4 COMO 9

DEFINIR SERVOPIN COMO 6

DEFINIR velocidad COMO 255

DEFINIR velocidadRetroceso COMO 255

DEFINIR velocidadGiro COMO 200

INICIALIZAR myServo

FUNCION moveForward()

ESTABLECER myServo A 90°

ESTABLECER IN3 A ALTO

ESTABLECER IN4 A BAJO

AJUSTAR ENA A velocidad

FIN FUNCION

FUNCION moveBack()

ESTABLECER myServo A 90°

ESTABLECER IN3 A BAJO

ESTABLECER IN4 A ALTO

AJUSTAR ENA A velocidadRetroceso

FIN FUNCION

```
FUNCION stop()
ESTABLECER IN3 A BAJO
ESTABLECER IN4 A BAJO
AJUSTAR ENAA 0
FIN FUNCION

FUNCION turnRight()
ESTABLECER myServo A 180°
ESTABLECER IN3 A ALTO
ESTABLECER IN4 A BAJO
AJUSTAR ENAA velocidadGiro
FIN FUNCION

FUNCION turnLeft()
ESTABLECER myServo A 0°
ESTABLECER IN3 A ALTO
ESTABLECER IN4 A BAJO
AJUSTAR ENAA velocidadGiro
FIN FUNCION

FUNCION leerSensores()
LEER distancia del sensor derecho
GUARDAR EN der
LEER distancia del sensor frontal
GUARDAR EN cen
LEER distancia del sensor izquierdo
GUARDAR EN izq
MOSTRAR der, cen E izq
FIN FUNCION

INICIALIZAR comunicación serial
```


CONFIGURAR los pines de los sensores
CONFIGURAR los pines del controlador L298N
CONECTAR el servomotor
COLOCAR el servo a 90°
LLAMAR moveForward()
MIENTRAS VERDADERO
LLAMAR leerSensores()
SI cen < 15
LLAMAR stop()
ESPERAR 1000 ms
LLAMAR moveBack()
ESPERAR 1300 ms
LLAMAR stop()
ESPERAR 500 ms
SI izq > der
LLAMAR turnLeft()
ESPERAR 800 ms
SINO SI der > izq
LLAMAR turnRight()
ESPERAR 800 ms
SINO
LLAMAR turnLeft()
ESPERAR 800 ms
FIN SI
SINO SI cen >= 15 Y cen <= 50
SI izq > der
LLAMAR turnLeft()

ESPERAR 900 ms

SINO SI der > izq

LLAMAR turnRight()

ESPERAR 800 ms

SINO

LLAMAR moveForward()

FIN SI

SINO SI der < 20

LLAMAR turnLeft()

ESPERAR 800 ms

SINO SI izq < 20

LLAMAR turnRight()

ESPERAR 800 ms

SINO

LLAMAR moveForward()

FIN SI

ESPERAR 100 ms

FIN MIENTRAS

FIN

8.3. Mejoramiento de la lógica

El 4 de junio analizamos la primera versión y decidimos modificarla porque necesitábamos una estrategia más adecuada para el recorrido de WRO.

En esta etapa se redujo la lógica de navegación a los sensores derecho e izquierdo, que serían utilizados para determinar las aperturas y los giros de la pista.

El cambio principal fue pasar de una lógica basada solamente en reaccionar ante obstáculos a una lógica capaz de identificar los giros del recorrido y contar las vueltas realizadas.

Inicio

...

Definir der, izq Como Real

Definir duracion Como Entero

Definir velRecta Como Entero

Definir velCurva Como Entero

Definir anguloRecto Como Entero

Definir anguloIzq Como Entero

Definir anguloDer Como Entero

Definir umbralGiro Como Real

Definir girosPorVuelta Como Entero

Definir vueltasMeta Como Entero

Definir contadorGiros Como Entero

Definir girosMeta Como Entero

Definir girandoAntes Como Logico

Definir girandoAhora Como Logico

Definir detenido Como Logico

Definir primerGiroIgnorado Como Logico

velRecta ← 220

velCurva ← 220

```
anguloRecto ← 90
anguloIzq ← 50
anguloDer ← 130
umbralGiro ← 55
girosPorVuelta ← 4
vueltasMeta ← 3
contadorGiros ← 0
girosMeta ← girosPorVuelta * vueltasMeta
girandoAntes ← Falso
detenido ← Falso
primerGiroIgnorado ← Falso
Configurar sensor derecho
Configurar sensor izquierdo
Configurar motor
Configurar servo
Mover servo a anguloRecto
Encender motor
Avanzar hacia adelante
Mientras detenido = Falso Hacer
  der ← LeerDistancia(sensorDerecho)
  izq ← LeerDistancia(sensorIzquierdo)
  Mostrar "Der: ", der, " cm"
  Mostrar "Izq: ", izq, " cm"
  Si der > umbralGiro Y izq <= umbralGiro Entonces
    Mover servo a anguloDer
    Velocidad del motor ← velCurva
    girandoAhora ← Verdadero
```

Sino Si $izq > umbralGiro$ Y $der \leq umbralGiro$ Entonces

Mover servo a $anguloIzq$

Velocidad del motor $\leftarrow velCurva$

$girandoAhora \leftarrow Verdadero$

Sino

Mover servo a $anguloRecto$

Velocidad del motor $\leftarrow velRecta$

$girandoAhora \leftarrow Falso$

FinSi

Si $girandoAhora = Verdadero$ Y $girandoAntes = Falso$ Entonces

Si $primerGiroIgnorado = Falso$ Entonces

$primerGiroIgnorado \leftarrow Verdadero$

Mostrar "Giro de arranque, no cuenta"

Sino

$contadorGiros \leftarrow contadorGiros + 1$

Mostrar "Giro detectado. Total: ", $contadorGiros$

Si $contadorGiros \geq girosMeta$ Entonces

$detenido \leftarrow Verdadero$

FinSi

FinSi

FinSi

$girandoAntes \leftarrow girandoAhora$

Esperar 100 milisegundos

FinMientras

Mover servo a $anguloRecto$

Detener motor

Velocidad del motor $\leftarrow 0$

...

Fin

8.4. Implementación en Arduino UNO

El 14 de junio transformamos el pseudocódigo funcional en un programa ejecutable en Arduino.

Esta fue una etapa importante porque permitió pasar de una estrategia teórica a una implementación que podía probarse directamente sobre el robot.

Se incorporaron las librerías:

```
#include <Servo.h>
```

```
#include <Arduino.h>
```

La primera versión del programa definió los pines de los sensores, controlador y servomotor.

También se crearon variables para las velocidades, ángulos, umbral de giro y número de vueltas.

Una de las funciones principales fue:

```
float leerDistancia(int trigPin, int echoPin)
```

Esta función permitió reutilizar el mismo procedimiento para cualquiera de los sensores ultrasónicos.

El proceso fue:

1. Colocar el pin TRIG en LOW.
2. Generar un pulso de aproximadamente 10 microsegundos.
3. Esperar la respuesta en el pin ECHO.
4. Medir la duración del pulso.
5. Convertir el tiempo en distancia.
6. Devolver el resultado en centímetros.

La conversión utilizada fue:

```
return duracion * 0.034 / 2;
```

El programa también contempló el caso en que no se recibiera una respuesta:

```
if (duracion == 0)
    return 400.0;
```

De esta manera, una ausencia de lectura no detenía inmediatamente el programa. Para organizar mejor el código se crearon funciones específicas para cada acción.

Movimiento recto

```
void irRecto() {
    myServo.write(anguloRecto);
    analogWrite(ENA, velRecta);
}
```

Esta función coloca el servomotor en 90° y establece la velocidad correspondiente al desplazamiento recto.

Giro a la izquierda

```
void girarIzquierda() {
    myServo.write(anguloIzq);
    analogWrite(ENA, velCurva);
}
```

El servo se mueve hasta el ángulo establecido para la izquierda.

Giro a la derecha

```
void girarDerecha() {
    myServo.write(anguloDer);
    analogWrite(ENA, velCurva);
}
```

El servo se mueve hasta el ángulo establecido para la derecha.

Detención

```
void detenerRobot() {
    myServo.write(anguloRecto);
    digitalWrite(IN3, LOW);
}
```

```

    digitalWrite(IN4, LOW);
    analogWrite(ENA, 0);
}

```

Esta función detiene el motor y devuelve el servomotor a la posición recta.

La creación de estas funciones hizo que el programa fuera más fácil de modificar y depurar, ya que cada acción física del robot estaba separada de la lógica de decisión.

```

#include
#include

const int TRIG_DER = 2;
const int ECHO_DER = 4;
const int TRIG_IZQ = 12;
const int ECHO_IZQ = 13;
float der, izq;

const int ENA = 11;
const int IN3 = 10;
const int IN4 = 9;
const int SERVOPIN = 6;
Servo myServo;

int velRecta = 220;
int velCurva = 220;
int anguloRecto = 90;
int anguloIzq = 50;
int anguloDer = 130;
float UMBRAL_GIRO = 55;
int GIROS_POR_VUELTA = 4;
int VUELTAS_META = 3;

```



```
int contadorGiros = 0;
int girosMeta = GIROS_POR_VUELTA * VUELTAS_META;
bool girandoAntes = false;
bool detenido = false;
bool primerGiroIgnorado = false;
void setup() {
  Serial.begin(9600);
  pinMode(TRIG_DER, OUTPUT);
  pinMode(ECHO_DER, INPUT);
  pinMode(TRIG_IZQ, OUTPUT);
  pinMode(ECHO_IZQ, INPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  myServo.attach(SERVOPIN);
  myServo.write(anguloRecto);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, velRecta);
}
float leerDistancia(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duracion = pulseIn(echoPin, HIGH, 25000);
```

```
if (duracion == 0)
return 400.0;
return duracion * 0.034 / 2;
}

void irRecto() {
myServo.write(anguloRecto);
analogWrite(ENA, velRecta);
}

void girarIzquierda() {
myServo.write(anguloIzq);
analogWrite(ENA, velCurva);
}

void girarDerecha() {
myServo.write(anguloDer);
analogWrite(ENA, velCurva);
}

void detenerRobot() {
myServo.write(anguloRecto);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);
analogWrite(ENA, 0);
}

void loop() {
if (detenido) {
detenerRobot();
return;
}
```

```
der = leerDistancia(TRIG_DER, ECHO_DER);
izq = leerDistancia(TRIG_IZQ, ECHO_IZQ);
Serial.print("Der: ");
Serial.print(der);
Serial.print(" cm | Izq: ");
Serial.print(izq);
Serial.println(" cm");
bool abrioDer = der > UMBRAL_GIRO;
bool abrioIzq = izq > UMBRAL_GIRO;
bool girandoAhora = false;
if (abrioDer && !abrioIzq) {
  girarDerecha();
  girandoAhora = true;
}
else if (abrioIzq && !abrioDer) {
  girarIzquierda();
  girandoAhora = true;
}
else {
  irRecto();
  girandoAhora = false;
}
if (girandoAhora && !girandoAntes) {
  if (!primerGirolIgnorado) {
    primerGirolIgnorado = true;
    Serial.println("Giro de arranque (no cuenta para el total)");
  }
}
```

```

else {
  contadorGiros++;
  Serial.print("Giro detectado. Total: ");
  Serial.println(contadorGiros);
  if (contadorGiros >= girosMeta) {
    detenido = true;
  }
}
}
}

girandoAntes = girandoAhora;
delay(100);
}

```

8.5. Calibración de velocidad

Después de implementar el código realizamos múltiples pruebas de pista.

Durante estas pruebas observamos que **la lógica del programa era funcional**, pero el comportamiento físico del robot podía variar dependiendo de la velocidad.

Por esta razón, entre el **15 y el 25 de junio**, nos concentramos principalmente en calibrar la velocidad.

Se probaron diferentes valores:

255 → 180 → 170 → 220

Cada configuración se probó en pista para observar principalmente:

- estabilidad del desplazamiento;
- comportamiento durante los giros;
- capacidad de reacción;
- trayectoria;
- recuperación después de una curva.

La velocidad **255** permitía un movimiento más rápido, pero presentaba pequeños problemas durante los giros.

Al reducir a **180** y posteriormente a **170**, conseguimos observar un comportamiento más controlado.

Sin embargo, después de comparar las diferentes configuraciones, determinamos que **220 era el valor más funcional para nuestro robot.**

Por ello, el valor final utilizado fue:

```
int velRecta = 220;
```

```
int velCurva = 220;
```

Esta etapa nos permitió comprobar que aumentar la velocidad no siempre significa mejorar el rendimiento. La velocidad tenía que ser compatible con la capacidad de los sensores y con el tiempo de respuesta necesario para realizar correctamente los giros.

Inicio

...

Definir der, izq Como Real

Definir duracion Como Entero

Definir velRecta Como Entero

Definir velCurva Como Entero

Definir anguloRecto Como Entero

Definir anguloIzq Como Entero

Definir anguloDer Como Entero

Definir umbralGiro Como Real

Definir girosPorVuelta Como Entero

Definir vueltasMeta Como Entero

Definir contadorGiros Como Entero

Definir girosMeta Como Entero

Definir girandoAntes Como Logico

Definir girandoAhora Como Logico

Definir detenido Como Logico

Definir primerGiroIgnorado Como Logico

velRecta $\leftarrow$ 220

velCurva $\leftarrow$ 220

anguloRecto $\leftarrow$ 90

anguloIzq $\leftarrow$ 50

anguloDer $\leftarrow$ 130

umbralGiro $\leftarrow$ 55

girosPorVuelta $\leftarrow$ 4

vueltasMeta $\leftarrow$ 3

contadorGiros $\leftarrow$ 0

girosMeta $\leftarrow$ girosPorVuelta * vueltasMeta

girandoAntes $\leftarrow$ Falso

detenido $\leftarrow$ Falso

primerGiroIgnorado $\leftarrow$ Falso

Configurar sensor derecho

Configurar sensor izquierdo

Configurar motor

Configurar servo

Mover servo a anguloRecto

Encender motor

Avanzar hacia adelante

Mientras detenido = Falso Hacer

der $\leftarrow$ LeerDistancia(sensorDerecho)

izq $\leftarrow$ LeerDistancia(sensorIzquierdo)

Mostrar "Der: ", der, " cm"

Mostrar "Izq: ", izq, " cm"

Si der > umbralGiro Y izq <= umbralGiro Entonces

Mover servo a anguloDer

Velocidad del motor $\leftarrow$ velCurva

girandoAhora $\leftarrow$ Verdadero

Sino Si izq > umbralGiro Y der <= umbralGiro Entonces

Mover servo a anguloIzq

Velocidad del motor $\leftarrow$ velCurva

girandoAhora $\leftarrow$ Verdadero

Sino

Mover servo a anguloRecto

Velocidad del motor $\leftarrow$ velRecta

girandoAhora $\leftarrow$ Falso

FinSi

Si girandoAhora = Verdadero Y girandoAntes = Falso Entonces

Si primerGiroIgnorado = Falso Entonces

primerGiroIgnorado $\leftarrow$ Verdadero

Mostrar "Giro de arranque, no cuenta"

Sino

contadorGiros $\leftarrow$ contadorGiros + 1

Mostrar "Giro detectado. Total: ", contadorGiros

Si contadorGiros >= girosMeta Entonces

detenido $\leftarrow$ Verdadero

FinSi

FinSi

FinSi

girandoAntes $\leftarrow$ girandoAhora

Esperar 100 milisegundos

FinMientras

Mover servo a anguloRecto

Detener motor

Velocidad del motor $\leftarrow 0$

...

Fin

Durante los días **7 y 14 de julio**, continuamos realizando pruebas con el código integrado al robot.

Durante esta etapa se realizaron cambios leves relacionados principalmente con la velocidad. Sin embargo, después de comparar nuevamente el comportamiento del robot, se decidió regresar al valor de **220**, que había demostrado ser el más funcional durante la calibración anterior.

Esto permitió mantener una configuración conocida antes de comenzar la siguiente etapa del proyecto.

8.6. Integración de la HuskyLens

El **6 de agosto**, después de recibir la confirmación de que habíamos avanzado a la etapa nacional, recibimos la posibilidad de trabajar con una **HuskyLens** para desarrollar la parte relacionada con los obstáculos.

El objetivo fue ampliar la capacidad de percepción del robot.

El **7 de agosto** retomamos el desarrollo del pseudocódigo para una versión capaz de trabajar con obstáculos y detección mediante visión.

Anteriormente habíamos comenzado a considerar esta posibilidad, pero el trabajo no pudo finalizarse porque en ese momento no contábamos con la cámara.

Una vez confirmada la disponibilidad de HuskyLens, pudimos completar la lógica necesaria para integrar la detección de colores a la estrategia de movimiento.

La nueva versión mantuvo la estructura principal de navegación del código anterior.

Esto fue importante porque **no reemplazamos completamente el sistema existente**.

En cambio, añadimos una nueva capa de percepción.

La velocidad, los ángulos, el umbral y el conteo de vueltas se conservaron:

- Velocidad recta: 220

- Velocidad curva: 220
- Servo recto: 90°
- Izquierda: 50°
- Derecha: 130°
- Umbral: 55
- 4 giros por vuelta
- 3 vueltas
- 12 giros objetivo

Esto permitió conservar una parte del código que ya había sido probada y añadir solamente las funciones necesarias para la nueva capacidad.

```
#include
```

```
#include
```

```
#include
```

```
#include
```

```
const int TRIG_DER = 2;
```

```
const int ECHO_DER = 4;
```

```
const int TRIG_IZQ = 12;
```

```
const int ECHO_IZQ = 13;
```

```
float der, izq;
```

```
const int ENA = 11;
```

```
const int IN3 = 10;
```

```
const int IN4 = 9;
```

```
const int SERVOPIN = 6;
```

```
Servo myServo;
```

```
HUSKYLENS huskylens;
```

```
int velRecta = 220;
```

```
int velCurva = 220;

int anguloRecto = 90;

int anguloIzq = 50;

int anguloDer = 130;

float UMBRAL_GIRO = 55;

int GIROS_POR_VUELTA = 4;

int VUELTAS_META = 3;

int contadorGiros = 0;

int girosMeta = GIROS_POR_VUELTA * VUELTAS_META;

bool girandoAntes = false;

bool detenido = false;

bool primerGiroIgnorado = false;

bool giroPorColor = false;

unsigned long inicioGiroColor = 0;

const unsigned long TIEMPO_MAX_GIRO = 1200;

const int COLOR_ROJO = 1;

const int COLOR_VERDE = 2;

void setup() {

  Serial.begin(9600);

  pinMode(TRIG_DER, OUTPUT);

  pinMode(ECHO_DER, INPUT);

  pinMode(TRIG_IZQ, OUTPUT);

  pinMode(ECHO_IZQ, INPUT);

  pinMode(ENA, OUTPUT);

  pinMode(IN3, OUTPUT);

  pinMode(IN4, OUTPUT);
```

```
myServo.attach(SERVOPIN);
myServo.write(anguloRecto);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);
analogWrite(ENA, velRecta);
Wire.begin();
if (huskylens.begin(Wire)) {
  Serial.println("HuskyLens conectada");
} else {
  Serial.println("Error de HuskyLens");
}
}

float leerDistancia(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duracion = pulseIn(echoPin, HIGH, 25000);
  if (duracion == 0)
    return 400.0;
  return duracion * 0.034 / 2;
}

int detectarColor() {
  if (!huskylens.request())
    return 0;
```

```

if (!huskylens.available())

return 0;

HUSKYLENSResult resultado = huskylens.read();

if (resultado.ID == COLOR_ROJO)

return COLOR_ROJO;

if (resultado.ID == COLOR_VERDE)

return COLOR_VERDE;

return 0;

}

void irRecto() {

myServo.write(anguloRecto);

analogWrite(ENA, velRecta);

}

void girarIzquierda() {

myServo.write(anguloIzq);

analogWrite(ENA, velCurva);

}

void girarDerecha() {

myServo.write(anguloDer);

analogWrite(ENA, velCurva);

}

void detenerRobot() {

myServo.write(anguloRecto);

digitalWrite(IN3, LOW);

digitalWrite(IN4, LOW);

analogWrite(ENA, 0);

```

```

}

void loop() {
  if (detenido) {
    detenerRobot();
    return;
  }

  int colorDetectado = detectarColor();
  if (!giroPorColor) {
    if (colorDetectado == COLOR_ROJO) {
      Serial.println("Rojo -> girar derecha");
      girarDerecha();
      giroPorColor = true;
      inicioGiroColor = millis();
    }

    else if (colorDetectado == COLOR_VERDE) {
      Serial.println("Verde -> girar izquierda");
      girarIzquierda();
      giroPorColor = true;
      inicioGiroColor = millis();
    }
  }

  if (giroPorColor) {
    if (colorDetectado == 0) {
      Serial.println("Color perdido -> control normal");
      giroPorColor = false;
      irRecto();
    }
  }
}

```

```

}

else if (millis() - inicioGiroColor >= TIEMPO_MAX_GIRO) {

Serial.println("Tiempo maximo de giro");

giroPorColor = false;

irRecto();

}

else {

if (colorDetectado == COLOR_ROJO) {

girarDerecha();

}

else if (colorDetectado == COLOR_VERDE) {

girarIzquierda();

}

}

delay(50);

return;

}

der = leerDistancia(TRIG_DER, ECHO_DER);

izq = leerDistancia(TRIG_IZQ, ECHO_IZQ);

Serial.print("Der: ");

Serial.print(der);

Serial.print(" cm | Izq: ");

Serial.print(izq);

Serial.println(" cm");

bool abrioDer = der > UMBRAL_GIRO;

bool abrioIzq = izq > UMBRAL_GIRO;

```

```
bool girandoAhora = false;
if (abrioDer && !abrioIzq) {
    girarDerecha();
    girandoAhora = true;
}
else if (abrioIzq && !abrioDer) {
    girarIzquierda();
    girandoAhora = true;
}
else {
    irRecto();
    girandoAhora = false;
}
if (girandoAhora && !girandoAntes) {
    if (!primerGiroIgnorado) {
        primerGiroIgnorado = true;
        Serial.println("Giro de arranque (no cuenta para el total)");
    }
    else {
        contadorGiros++;
        Serial.print("Giro detectado. Total: ");
        Serial.println(contadorGiros);
        if (contadorGiros >= girosMeta) {
            detenido = true;
        }
    }
}
```

```
}
```

```
girandoAntes = girandoAhora;
```

```
delay(100);
```

```
}
```


Impresión 3D

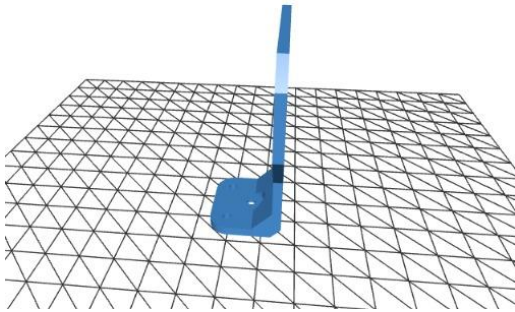


Figura 9.1 Del diseño 3D para el soporte de los sensores ultrasónicos.

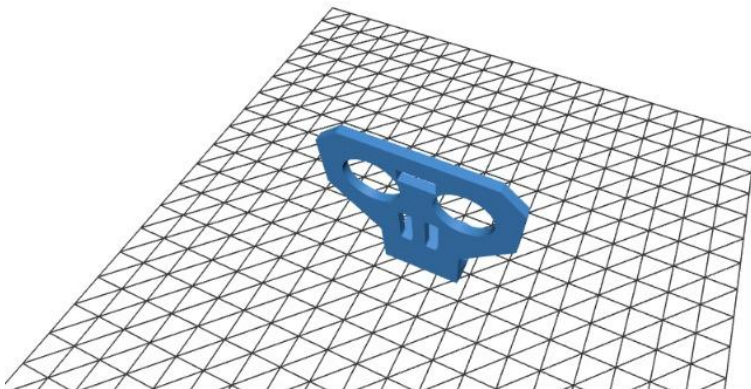


Figura 9.2

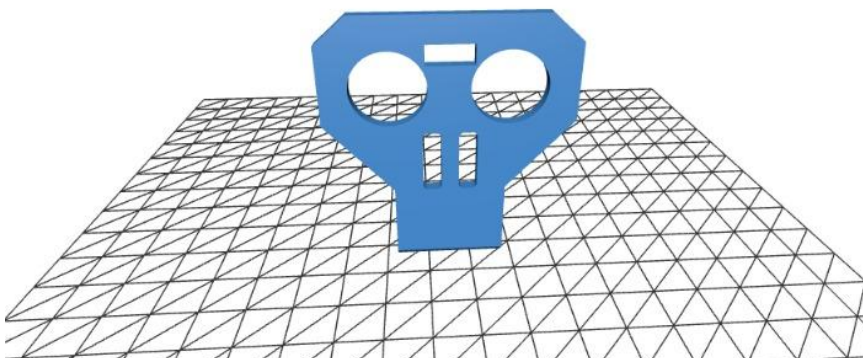
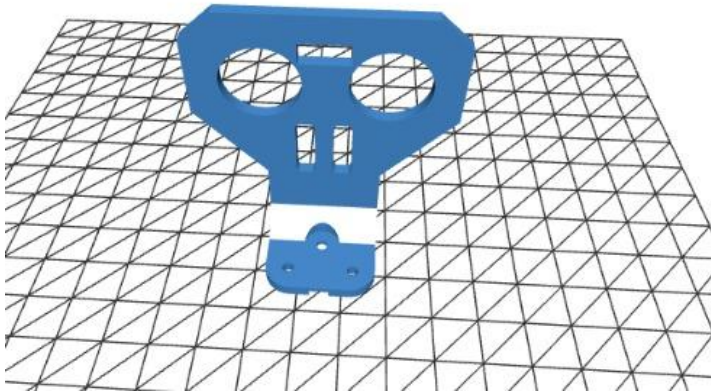
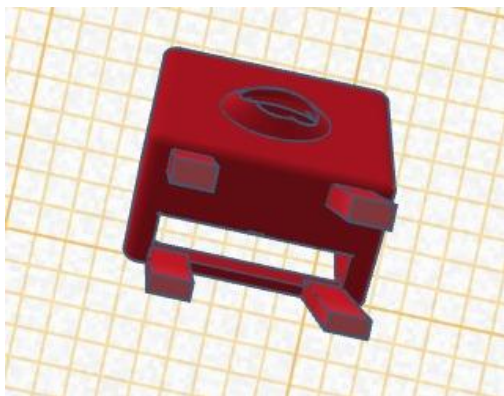


Figura 9.3**Figura 9.4 Diseño 3D soporte de la camara Huskylens parte de abajo del diseño.****Figura 9.5. Diseño 3D parte frontal con orificio para el lente de la camara**

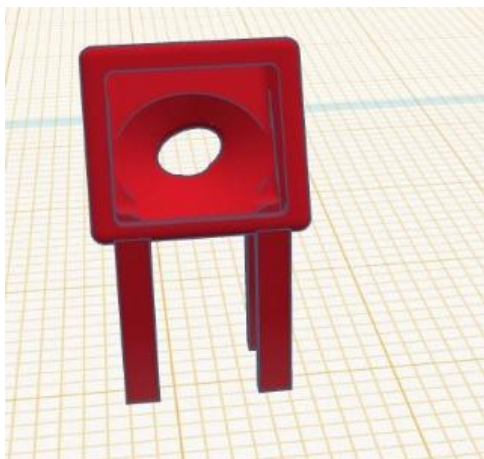
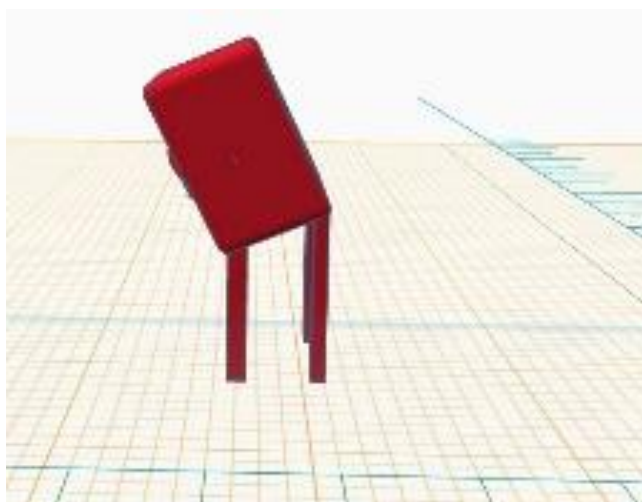
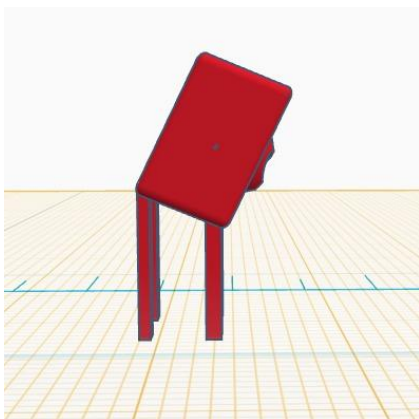


Figura 9.6. Diseño 3D costados del diseño



HuskyLens

```
#include
#include
#include //librerias
const int TRIG_DER = 2;
const int ECHO_DER = 4; //pines de los sensores
const int TRIG_IZQ = 12;
const int ECHO_IZQ = 13;
float der, izq; //Sentido
const int ENA = 11;
const int IN3 = 10;
const int IN4 = 9; //Pines del controlador
const int SERVOPIN = 6; //servo pin
Servo myServo;
int velRecta = 220; //velocidades 220
int velCurva = 220;
int anguloRecto = 90; //direcciones
int anguloIzq = 50;
int anguloDer = 130;
float UMBRAL_GIRO = 55; //distancia
int GIROS_POR_VUELTA = 4;
int VUELTAS_META = 3;
int contadorGiros = 0; //contador de giros
int girosMeta = GIROS_POR_VUELTA * VUELTAS_META;
bool girandoAntes = false;
bool detenido = false;
```

```
bool sentidoDefinido = false; // true una vez que ya sabemos hacia dónde gira la
pista
```

```
bool sentidoHorario = true; // true = horario (usa sensor derecho), false =
antihorario (usa sensor izquierdo)
```

```
// --- NUEVO: el primer giro que se detecta al arrancar no cuenta ---
```

```
// Es solo el ajuste inicial de posición (el robot empieza cerca de la
```

```
// primera esquina), no un giro real de una vuelta completa. Si se
```

```
// cuenta, al final falta exactamente ese giro para terminar el
```

```
// recorrido y cerrar bien hasta la meta/punto de partida.
```

```
bool primerGiroIgnorado = false;
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  pinMode(TRIG_DER, OUTPUT);
```

```
  pinMode(ECHO_DER, INPUT);
```

```
  pinMode(TRIG_IZQ, OUTPUT);
```

```
  pinMode(ECHO_IZQ, INPUT);
```

```
  pinMode(ENA, OUTPUT);
```

```
  pinMode(IN3, OUTPUT);
```

```
  pinMode(IN4, OUTPUT);
```

```
  myServo.attach(SERVOPIN);
```

```
  myServo.write(anguloRecto);
```

```
  digitalWrite(IN3, LOW);
```

```
  digitalWrite(IN4, HIGH);
```

```
  analogWrite(ENA, velRecta);
```

```
}
```

```
float leerDistancia(int trigPin, int echoPin) {
```

```
  digitalWrite(trigPin, LOW);
```

```

delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
long duracion = pulseIn(echoPin, HIGH, 25000);
if (duracion == 0)
return 400.0;
return duracion * 0.034 / 2;
}
void irRecto() {
myServo.write(anguloRecto);
analogWrite(ENA, velRecta);
}
void girarIzquierda() {
myServo.write(anguloIzq);
analogWrite(ENA, velCurva);
}
void girarDerecha() {
myServo.write(anguloDer);
analogWrite(ENA, velCurva);
}
void detenerRobot() {
myServo.write(anguloRecto);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);
analogWrite(ENA, 0);
}

```

```

void loop() {
  if (detenido) {
    detenerRobot();
    return;
  }

  der = leerDistancia(TRIG_DER, ECHO_DER);
  //delay(50);

  izq = leerDistancia(TRIG_IZQ, ECHO_IZQ);
  //delay(50);

  Serial.print("Der: ");
  Serial.print(der);
  Serial.print(" cm | Izq: ");
  Serial.print(izq);
  Serial.println(" cm");

  bool abrioDer = der > UMBRAL_GIRO;
  bool abrioIzq = izq > UMBRAL_GIRO;
  bool girandoAhora;

  if (!sentidoDefinido) {
    if (abrioDer && !abrioIzq) {
      sentidoDefinido = true;
      sentidoHorario = true; // se abrió a la derecha -> pista en sentido horario
      Serial.println(">> Sentido de pista definido: HORARIO");
    }

    else if (abrioIzq && !abrioDer) {
      sentidoDefinido = true;
      sentidoHorario = false; // se abrió a la izquierda -> pista antihoraria
      Serial.println(">> Sentido de pista definido: ANTIHORARIO");
    }
  }
}

```

```

}
else if (abrioDer && abriolzq) {
// si se abren los dos a la vez, se decide por el que esté más despejado
sentidoDefinido = true;
sentidoHorario = (der > izq);
Serial.print(">> Sentido de pista definido (ambos abiertos): ");
Serial.println(sentidoHorario ? "HORARIO" : "ANTIHORARIO");
}
}
if (abrioDer && !abriolzq) {
girarDerecha();
girandoAhora = true;
}
else if (abriolzq && !abrioDer) {
girarIzquierda();
girandoAhora = true;
}
else if (abrioDer && abriolzq) {
if (der > izq)
girarDerecha();
else
girarIzquierda();
girandoAhora = true;
}
else {
irRecto();
girandoAhora = false;
}

```



```

}
if (girandoAhora && !girandoAntes) { //contador de los giros
if (!primerGiroIgnorado) {
// este es el giro de arranque: se ejecuta, pero no cuenta
primerGiroIgnorado = true;
Serial.println("Giro de arranque (no cuenta para el total)");
}
else {
contadorGiros++;
Serial.print("Giro detectado. Total: ");
Serial.println(contadorGiros);
if (contadorGiros >= girosMeta) {
detenido = true;
}
}
}
girandoAntes = girandoAhora;
delay(100);
}

```

El 10 de agosto convertimos el pseudocódigo de la nueva estrategia en código Arduino.

La principal diferencia respecto a la versión anterior fue la incorporación de:

```

#include <Wire.h>

#include <HUSKYLENS.h>

y:

HUSKYLENS huskylens;

```

Esto permitió establecer la comunicación entre Arduino y HuskyLens mediante I²C.

Inicialización de HuskyLens

Dentro de setup() se agregó:

```
Wire.begin();
```

```
if (huskylens.begin(Wire)) {
    Serial.println("HuskyLens conectada");
} else {
    Serial.println("Error de HuskyLens");
}
```

Esta parte permite comprobar si la comunicación con HuskyLens se establece correctamente.

Si la conexión funciona, el programa muestra:

```
"HuskyLens conectada"
```

Si existe un problema:

```
"Error de HuskyLens"
```

Esto facilita el diagnóstico durante las pruebas.

Para separar la detección de color de la lógica principal se creó:

```
int detectarColor()
```

La función solicita información a HuskyLens:

```
huskylens.request()
```

y posteriormente verifica si existe información disponible.

Cuando se obtiene un resultado, el programa analiza el identificador:

```
if (resultado.ID == COLOR_ROJO)
```

```
    return COLOR_ROJO;
```

```
if (resultado.ID == COLOR_VERDE)
```

```
    return COLOR_VERDE;
```

Por lo tanto, se establecieron:

```
const int COLOR_ROJO = 1;
```

```
const int COLOR_VERDE = 2;
```

La estrategia desarrollada fue:

Rojos → giro derecha

Verde → giro izquierda

Esto permitió incorporar información visual a la toma de decisiones del robot.

Control del giro mediante color

Se agregó una nueva variable:

```
bool giroPorColor = false;
```

Esta variable permite saber si el robot se encuentra actualmente ejecutando un giro provocado por la detección de un color.

También se añadió:

```
unsigned long inicioGiroColor = 0;
```

y:

```
const unsigned long TIEMPO_MAX_GIRO = 1200;
```

Esto estableció un tiempo máximo para el giro generado por color.

Integración de los dos sistemas — 15 de agosto

El 15 de agosto comenzó una de las etapas más importantes: unir los dos programas.

Hasta ese momento teníamos dos sistemas funcionales:

Sistema 1

Navegación mediante sensores ultrasónicos.

Sistema 2

Detección de colores mediante HuskyLens.

El objetivo fue crear un único programa que pudiera manejar ambas funciones sin generar conflictos.

Durante esta integración revisamos especialmente:

- Pines.
- Librerías.
- Variables.
- Funciones.
- Lectura de sensores.
- Comunicación con HuskyLens.
- Control del servomotor.
- Control de motores.
- Conteo de giros.
- Condiciones de finalización.

Uno de los puntos principales fue comprobar que la incorporación de HuskyLens no alterara accidentalmente la configuración utilizada por los sensores o el controlador.

Código integrado — 16 de agosto

El 16 de agosto obtuvimos una versión integrada que será utilizada como base para las próximas pruebas y preparación para la competencia.

Una modificación importante respecto al código anterior fue que el programa ya no depende únicamente de detectar una apertura para saber cómo recorrer la pista.

Ahora incorpora una variable para definir el sentido:

```
bool sentidoDefinido = false;
```

```
bool sentidoHorario = true;
```

Esto permite que el robot determine inicialmente hacia qué sentido está orientada la pista.

Determinación del sentido de la pista

El programa analiza las aperturas detectadas por los sensores.

Si:

```
abrioDer && !abrioIzq
```

se interpreta que la pista está orientada en sentido horario.

Si:

```
abrioIzq && !abrioDer
```

se interpreta como sentido antihorario.

Cuando ambos lados están abiertos, se compara cuál presenta una mayor distancia:

```
sentidoHorario = (der > izq);
```

De esta manera, el programa puede establecer una dirección de recorrido a partir de la información obtenida durante el inicio.

Corrección del primer giro

En la versión final se mantuvo una condición importante:

```
bool primerGiroIgnorado = false;
```

El primer giro detectado al comenzar el recorrido se ejecuta pero no se cuenta.

Esto se debe a que el robot comienza cerca de la primera esquina y ese movimiento inicial corresponde al ajuste de posición, no a un giro completo que deba formar parte del conteo de vueltas.

La lógica evita que el robot termine el recorrido antes de tiempo o que llegue al punto final con un giro pendiente.

Organización del equipo

Integrantes		
Makenzie Agrazal	Electronica y documentacion	Ensamblaje del chasis, conexiones, cronología, reuniones, prácticas, fotografías y Diario de Ingeniería completo
Cesar Yau	Diseño 3D y programación	Base HuskyLens, soportes, apoyo en programación y mejora del código
David Garibaldy	Programación	Código Arduino, depuración, estrategia de navegación y programación de cámara

Trabajo colaborativo: Aunque cada integrante tuvo responsabilidades específicas dentro del proyecto, las actividades de validación y desarrollo se realizaron de manera conjunta. Los tres integrantes participaron en las pruebas de pista, selección de componentes, toma de decisiones, análisis del comportamiento del robot y desarrollo de mejoras. Después de cada prueba se discutían los resultados obtenidos, se identificaban los problemas y se evaluaban posibles soluciones. Posteriormente, las modificaciones se volvían a probar en pista para comprobar su efectividad. De esta manera, las decisiones finales del equipo estuvieron basadas

en la observación del funcionamiento real del robot y en el aporte de los tres integrantes.

Makenzie Agrazal Velasquez se encargó de la electrónica base del prototipo. Se encargó del montaje y organización de los principales componentes electrónicos, realizando las conexiones necesarias para integrar el Arduino, los sensores, el controlador de motores, el servomotor y la alimentación del sistema.

Su trabajo en electrónica incluyó la revisión de las conexiones, identificación de los componentes, organización del cableado y comprobación de que las diferentes partes del circuito estuvieran correctamente conectadas

También participó directamente en el ensamblaje del chasis, integrando físicamente la electrónica con la estructura del robot.

Además de la parte electrónica, Makenzie fue responsable de toda la documentación del proyecto

Entre sus responsabilidades estuvieron:

- Desarrollo y montaje de la electrónica base del robot.
- Realización de las conexiones entre Arduino, sensores, controlador de motores y servomotor.
- Organización del cableado del prototipo.
- Revisión de conexiones durante las pruebas.
- Identificación y seguimiento de problemas relacionados con las c
- Integración de la electrónica con el chasis.
- Ensamblaje de la estructura física del robot.
- Apoyo en las pruebas de funcionamiento.
- Elaboración completa del Diario de Ingeniería.
- Registro de las diferentes etapas de desarrollo.
- Administración de la cronología del proyecto.
- Registro de las reuniones del equipo.
- Organización de las jornadas de práctica.
- Registro de las pruebas realizadas.
- Recopilación y organización de fotografías y evidencias.

- Documentación de los problemas encontrados y las soluciones aplicadas.
- Seguimiento de las modificaciones realizadas al robot.
- Organización de la información necesaria para la documentación de WRO.

Su participación en electrónica también estuvo relacionada directamente con el diagnóstico de los problemas del robot. Por ejemplo, cuando aparecieron inconsistencias en las lecturas de los sensores, se revisaron las conexiones, el cableado y los componentes antes de concluir que el problema podía estar relacionado con el procesamiento de las lecturas.

David Garibaldy estuvo principalmente encargado de la programación del robot, trabajando conjuntamente con Cesar en diferentes etapas de desarrollo y mejora del código.

Su trabajo consistió en convertir la lógica planteada en los

Entre sus responsabilidades estuvieron:

- Desarrollo del código principal del robot.
- Conversión del pseudocódigo a Arduino Code.
- Programación de los movimientos.
- Programación de la lectura de los sensores.
- Desarrollo de las condiciones para los giros.
- Implementación del conteo de giros.
- Programación de la condición de finalización.
- Ajuste de parámetros de velocidad.
- Depuración del programa.
- Análisis de errores durante las pruebas.
- Modificación de la lógica de navegación.
- Trabajo conjunto con Cesar para mejorar el código.
- Programación relacionada con la cámara durante la etapa en que se contó con este recurso.
- Participación en las pruebas de integración del sistema de visión.

David tuvo que realizar múltiples pruebas para comprobar que la lógica programada coincidiera con el comportamiento físico esperado. Esto incluyó analizar cuándo el robot debía avanzar, girar, corregir su trayectoria y detenerse.

Responsabilidad principal: programación, desarrollo y depuración del código Arduino, además de programación de la cámara y colaboración en las mejoras del software.

Cesar Yau Feng estuvo principalmente encargado del diseño 3D

Su trabajo fue especialmente importante durante las etapas en las que se necesitaron soportes específicos para los sensores y posteriormente una base para HuskyLens.

Entre sus responsabilidades estuvieron:

- Diseño de piezas mediante herramientas de diseño 3D.
- Desarrollo de soportes para componentes del robot.
- Diseño de la base para HuskyLens.
- Modificación de piezas cuando las primeras versiones no cumplían adecuadamente su función.
- Apoyo en la integración física de los componentes.
- Participación en las pruebas de las piezas diseñadas.
- Apoyo a David en la programación.
- Análisis de errores encontrados durante las pruebas.
- Participación en la mejora de la lógica del código.
- Propuesta de modificaciones para mejorar el comportamiento del robot.

Una de sus contribuciones más importantes fue el diseño de soluciones para mantener los componentes correctamente posicionados. Esto fue relevante porque durante las pruebas se comprobó que un sensor que se desplazara o vibrara podía afectar las lecturas y, por consecuencia, las decisiones del programa.

Posteriormente, Cesar participó en el diseño de la base 3D para HuskyLens, permitiendo que el dispositivo pudiera montarse de forma más estable y repetible sobre el robot.

Conclusión

El desarrollo de nuestro carrito demostró que la ingeniería de un sistema autónomo depende de la integración entre mecánica, electrónica, sensores y software. El proyecto comenzó con una plataforma que necesitaba ajustes de trayectoria y terminó con una solución mucho más controlada gracias a una sucesión de pruebas y decisiones fundamentadas en observaciones reales.

Uno de los aprendizajes más importantes fue que un problema observado no necesariamente tiene su origen allí. Las lecturas inestables de los ultrasónicos llevaron primero a revisar el cableado y los sensores, pero finalmente se comprobó que el procesamiento de las mediciones era una parte importante del problema. El cambio de el sensor ultrasonico y los ajustes de los angulos mejoró la estabilidad de las decisiones. Del mismo modo, el rediseño del soporte 3D mostró que la mecánica puede afectar directamente la calidad de la percepción.

La calibración de velocidad y dirección permitió encontrar un equilibrio entre rapidez y capacidad de reacción. El conteo de giros añadió una condición de finalización autónoma. Posteriormente, la incorporación de HuskyLens y el diseño de una base 3D ampliaron las posibilidades de percepción y montaje. Sin embargo, la experiencia regional fue especialmente significativa: al no contar con la cámara, el equipo pudo continuar con una estrategia sin ella. Esto convirtió una limitación de recursos en una prueba de robustez del diseño.

Finalmente, la corrección específica del sentido horario representó la etapa final de un proceso iterativo en el que el equipo comparó el comportamiento real del robot con el comportamiento esperado. Más que buscar una única versión perfecta desde el principio, el proyecto avanzó mediante diagnóstico, hipótesis, pruebas, correcciones y documentación.

El resultado más importante del proyecto no es únicamente el robot físico. Es el conocimiento acumulado sobre por qué se tomaron las decisiones, qué problemas aparecieron, qué soluciones funcionaron y cómo puede otra persona comprender y reproducir el sistema.